1 Estimator Infrastructure: WLS Seeding, State Initialisation, and Pivot Management

The Kalman filter of the filtering document is a *local* linear estimator: every update assumes the state lies inside a small neighbourhood of the linearisation point, and every dynamic state, meaning ambiguity, per-satellite ionosphere, receiver code bias, satellite bias, or inter-system offset (the state document), must be born with a value and a covariance before it can be updated. This document covers the *support* machinery that supplies those preconditions: the weighted-least-squares (WLS) epoch solve that drops the receiver inside the linearisation radius (Section 1.1); the first-epoch residual-differencing seeds that give each newly created state a physically informed prior (Section 1.2); and the between-satellite pivot selection and disturbance logic that keep the single-difference (SD) reference frame stable across an arc (Section 1.3). None of this is filtering proper; it is the scaffolding that makes the filter's local-linear assumptions valid at the first epoch and after every discontinuity.

The lossless covariance re-referencing algebra (the $\boldsymbol{T}\boldsymbol{P}\boldsymbol{T}^{\mathrm{T}}$ S-transform that re-anchors a group onto a new pivot) is deliberately *out of scope* here: it is derived in the differencing document and implemented once in `include/filter/CovarianceReReference.hpp`, which both the single-receiver and network handover paths call. This section names that seam where the pivot logic invokes it and otherwise refers the reader to the differencing derivation.

1.1 The WLS epoch estimator: receiver-clock seed and post-gap re-seed

Before the filter can run, the receiver position must be close enough to truth that the geometry Jacobian linearised about it is valid to sub-decimetre. Broadcast pseudoranges alone fix this: a GPS-only, code-only iterated Gauss–Newton solve over the compact core state

$$\boldsymbol{\xi} = \begin{bmatrix} \boldsymbol{r}^{\mathrm{T}} & c\,\mathrm{d}t_r & \mathrm{ZWD} \end{bmatrix}^{\mathrm{T}} \in \mathbb{R}^5, \tag{1}$$

with $\boldsymbol{r} \in \mathbb{R}^3$ the ECEF receiver position, $c\,\mathrm{d}t_r$ the receiver clock in metres, and ZWD the zenith wet delay (`src/core/math/WlsEstimator.cpp`). At least `kCoreSize` $= 5$ code observations are required before the solve is attempted; the iteration cap is `kMaxIter` $= 5$.

For each code measurement i the prefit residual removes every modelled term and the current state's contribution,

$$\widetilde{z}_i = P_i - \left[\, P_i^{\mathrm{mod}} + (\rho_i - \rho_i^0) + c\,\mathrm{d}t_r + m_w(i)\,\mathrm{ZWD} \,\right], \tag{2}$$

where P_i^{mod} is the modelled pseudorange, $\rho_i - \rho_i^0$ the change in geometric range relative to the range at the iteration's linearisation point, and $m_w(i)$ the wet mapping function (`WlsEstimator::predictObs`, evaluated once per row from the design-row loop of `WlsEstimator::estimate`). The geometric range is *re-evaluated every iteration* about the running position estimate inside that loop, so this is a genuine Gauss–Newton relinearisation rather than a single-shot least-squares fit.

The design row for satellite i is the negated line-of-sight stacked with the clock and wet-mapping partials, assembled in the same loop:

$$\boldsymbol{A}_i = \begin{bmatrix} -\boldsymbol{e}_i^{\mathrm{T}} & 1 & m_w(i) \end{bmatrix} = \begin{bmatrix} -e_x & -e_y & -e_z & 1 & m_w(i) \end{bmatrix}, \tag{3}$$

with $\boldsymbol{e}_i$ the receiver-to-satellite unit vector (the same line-of-sight that drives the observation geometry). Stacking all rows into $\boldsymbol{A}$, the prefit residuals into $\widetilde{z}$, and the inverse measurement

variances onto the diagonal weight $\boldsymbol{W}$ (clamped from below by `kMinVariance` $= 10^{-6}$ in that same loop), the normal-equations step is the boxed Gauss–Newton update

$$\boxed{\Delta\boldsymbol{\xi} = \left(\boldsymbol{A}^{\mathrm{T}}\boldsymbol{W}\,\boldsymbol{A}\right)^{-1}\boldsymbol{A}^{\mathrm{T}}\boldsymbol{W}\,\widetilde{z},} \tag{4}$$

applied once per Gauss–Newton iteration of `WlsEstimator::estimate`. Iteration terminates on a position increment $\|\Delta\boldsymbol{r}\| <$ `kTol` $= 0.1\,\mathrm{m}$ or at `kMaxIter`. The normal matrix is guarded against rank deficiency: if

$$\det\!\left(\boldsymbol{A}^{\mathrm{T}}\boldsymbol{W}\,\boldsymbol{A}\right) \le 10^{-6} \tag{5}$$

the solve aborts and the epoch is left un-seeded (the determinant guard immediately preceding the step), which protects against the degenerate single-constellation, near-coplanar geometry that would otherwise produce a wild position step.

On convergence the estimator returns both the state $\boldsymbol{\xi} = [\boldsymbol{r}^{\mathrm{T}},\ c\,\mathrm{d}t_r,\ \mathrm{ZWD}]^{\mathrm{T}}$ and its formal covariance, the inverse normal matrix

$$\boldsymbol{P}_{\boldsymbol{\xi}} = \left(\boldsymbol{A}^{\mathrm{T}}\boldsymbol{W}\,\boldsymbol{A}\right)^{-1} \tag{6}$$

(the result assembly at the tail of `WlsEstimator::estimate`). Of this pair the filter consumes only the *clock*: `Application::handleWlsSeeding` writes $c\,\mathrm{d}t_r$ into the receiver-clock state, and only when one of three triggers fires: the first epoch, a data gap, or a clock *jump* of at least `kClockJumpThreshSec` $= 0.5\,\mathrm{ms}$ between the WLS and filter clock estimates (in practice the jump, not the gap, is the dominant trigger). The position, the zenith wet delay, and the covariance (6) are computed but not consumed as filter seeds: the position prior is the RINEX-header `approx_position` plus the antenna offset, and the WLS position re-seeds the filter only on a kinematic (non-IMU) long-gap cold start (`Application::handleGap`); the ZWD estimate reaches only a trace-CSV column; and the formal covariance has no filter consumer at all. The whole seeding path is skipped under BSSD, where the filter carries no receiver-clock state for the WLS clock to seed (`handleWlsSeeding` early-returns on `!hasClock()`). The clock seed's recurring role is *post-gap re-seeding*: the receiver clock is the most volatile state across a data gap (an unsteered oscillator can drift arbitrarily while no measurements constrain it), so after a gap the WLS clock estimate re-anchors $c\,\mathrm{d}t_r$ before the phase filter resumes, rather than letting the filter chase a stale clock prior through a sequence of large innovations.

1.2 First-epoch state seeding by residual differencing

The WLS solve of Section 1.1 seeds the receiver clock (and, on a kinematic cold start, position); it does not touch the *differential* states. Every such state (inter-system bias, phase clock, uncombined receiver code bias, per-satellite ionosphere, carrier ambiguity) is created empty and must be seeded from the measurement residuals. The unifying idea (`src/core/measurement/StateInitializer.cpp`) is that each such state is precisely the quantity left in a particular *difference* of residuals once all already-modelled terms are removed; estimating it is a matter of forming that difference and taking a robust (median) central value. The shared prefit residual is

$$\widetilde{z}^{\,i}_{r,s} = \Phi/P^{i}_{r,s} - \left(\text{modelled} + c\,\mathrm{d}t_r + m_w\,\mathrm{ZWD} + \mathrm{ISB} + g_{\mathrm{trop}} + \alpha_s\widehat{I}^{\,i} + \beta_s b_r + b^{i}\right), \tag{7}$$

the observation minus every term the model can already account for, evaluated at the current state (`StateInitializer::computeRawResidual`, which also forms the tropospheric gradient projection $g_{\mathrm{trop}} = m_g(g_N\cos\mathrm{az} + g_E\sin\mathrm{az})$). The five *differential* seeds (a)–(e) below are differences of (7); a sixth, network-only pre-seed (f) anchors the receiver clock itself.

(a) Inter-system bias. Each non-GPS constellation g carries a receiver clock offset relative to the GPS time scale (the inter-system offset state). Because the GPS clock is the datum that the WLS step has already absorbed into $c\,\mathrm{d}t_r$, the ISB of g is the constellation-median code residual minus the GPS-median residual:

$$\boxed{\mathrm{ISB}_g = \mathrm{med}_{i\in g}\ \widetilde{z}^i - \mathrm{med}_{i\in \mathrm{GPS}}\ \widetilde{z}^i} \tag{8}$$

(`StateInitializer::initializeInterSystemBiases`). Medians, not means, are used throughout so a single gross code outlier cannot capsize the seed. A hard guard requires at least one GPS code observation at the seeding epoch (the GPS-median guard in that same routine): if GPS is absent it returns `false` and *retries the next epoch* rather than latching the ISB states as initialised against an arbitrary datum. This avoids the failure mode where a constellation that happens to acquire first becomes the implicit zero, and a GPS satellite rising later is then forced to wear the whole offset.

(b) Phase clock. In the uncombined model the phase observations see a receiver clock that differs from the code clock by the receiver phase–code hardware delay (the phase-clock state). Its seed is the median over satellites carrying *both* a code and a phase observation of the code-minus-phase residual:

$$\delta_\Phi = \mathrm{med}_i\ \left(\widetilde{z}^i_P - \widetilde{z}^i_\Phi\right) \tag{9}$$

(`StateInitializer::initializePhaseClockOffset`). The seed's covariance is then set from the code noise, tightened relative to the (looser) default initial standard deviation but *independent* of the n differences that formed the median (the covariance-seeding tail of that routine):

$$\sigma_{\delta_\Phi} = \max\!\Big(\tfrac{\sigma_P}{\sqrt{n}},\ \sigma_P\Big), \qquad \boldsymbol{P}(\delta_\Phi,\delta_\Phi) = \sigma^2_{\delta_\Phi}, \tag{10}$$

with the corresponding row and column of $\boldsymbol{P}$ first zeroed so the seeded state starts uncorrelated. The max against σ_P makes the $\sigma_P/\sqrt{n}$ branch unreachable, since $\sigma_P/\sqrt{n} \le \sigma_P$ for every $n \ge 1$, so the seeded variance is σ_P^2 regardless of n: a conservative floor at the single-measurement code noise, not a $1/\sqrt{n}$ promise.

(c) Uncombined receiver code bias / IFB. In the uncombined PPP-RTK model each non-reference code band carries a receiver differential code bias (DCB) or inter-frequency bias (IFB) state relative to the constellation's BIA clock-reference band (the code-bias state). Forming the residual difference between band j and the reference band, removing the differential ionospheric contribution (the two bands scale the same slant ionosphere by different α), and dividing by the difference of receiver-bias Jacobian coefficients gives

$$\boxed{\widehat{b}_{r,j} = \frac{\Delta\widetilde{z}_j - \Delta(\alpha_j\,\widehat{I})}{\Delta\beta_j} = \frac{(\widetilde{z}_j - \widetilde{z}_{\mathrm{ref}}) - (\alpha_j - \alpha_{\mathrm{ref}})\,\widehat{I^i}}{\beta_j - \beta_{\mathrm{ref}}}} \tag{11}$$

(`StateInitializer::initializeCodeBiases`), median-aggregated over satellites and subject to the same $|\widehat{b}| > 200\,\mathrm{m}$ reject as the ionosphere seed below. The Jacobian coefficient β_s depends on the bias parameterisation (`StateInitializer::resolveReceiverBias`): for a pure IFB state it is the unit coefficient $\beta = 1$, whereas for a geometry-free / ionosphere-free DCB state against reference bands of frequencies f_1, f_2 it is

$$\mu_1 = 1, \quad \mu_2 = \frac{f_1^2}{f_2^2}, \quad \Delta\mu = \mu_1 - \mu_2, \qquad \beta_j = \begin{cases} \mu_1/\Delta\mu & j = \text{band 1} \\ \mu_2/\Delta\mu & j = \text{band 2}, \end{cases} \tag{12}$$

guarded by $|\Delta\mu| \geq 10^{-12}$ to avoid dividing by a near-degenerate frequency separation.

A third, higher-priority branch precedes both cases for GLONASS: under `enable_glonass_ifb` each FDMA band (G1, G2) carries one bias state *per FCN slot* k with $\beta = 1$ (`StateKeys::makeGlonassPerSlotIfbKey`), because the per-slot carrier $f = f_{\text{base}} + k\,\Delta f$ gives a slot-dependent receiver group delay that a single per-band state cannot represent; the central channel $k = 0$ is the held datum (no state) and the CDMA bands (G3/G4/G6) fall through to the legacy per-band treatment above. This branch runs before the clock-reference lookup because GLONASS usually ships no clock-reference pair in the `.BIA` product and would otherwise early-return before reaching any IFB logic; with the flag off a non-reference GLONASS band carries no IFB state at all (the per-slot and non-reference-band branches of `resolveReceiverBias`).

(d) Per-satellite slant ionosphere. Each tracked satellite's slant ionospheric delay (the per-satellite iono state) is recovered from the geometry-free combination of two code bands. With ionospheric scale factors $\alpha_s = f_{\text{ref}}^2 / f_s^2$ referenced to GPS L1 (the iono-scaling convention), the seed is

$$\boxed{\widehat{I}^i = \frac{(P_1 - P_1^{\text{mod}}) - (P_2 - P_2^{\text{mod}})}{\alpha_1 - \alpha_2}} \tag{13}$$

(`StateInitializer::computeIonosphereSeed`), where P_k^{mod} is the modelled pseudorange on band k. Two structural guards apply: the denominator must satisfy $|\alpha_1 - \alpha_2| \geq 10^{-6}$ (the denominator guard in that routine), and the seed is rejected unless it is finite and $|\widehat{I}^i| < 200\,\text{m}$ (its magnitude reject). The 200 m reject is a physical sanity bound: a slant ionospheric delay larger than this is never real and signals a band mis-pairing or a gross code outlier, which must not be latched into a per-satellite state. The seed additionally applies two BIA-coverage gates so the differenced code is bias-consistent: *presence* (a `.BIA` entry exists on the band, with the zero-valued BeiDou C2I/C6I entries treated as missing), and, outside the network producer, which keeps presence-only semantics, coverage *honesty* (`bias_coverage_requires_apply`), which skips the seed when an entry is loaded but not *applied* to $(P - \text{mod})$ so the raw hardware bias still survives. The honesty gate matters precisely because the failure it blocks, a zero-valued BeiDou entry injecting 30–60 m, sits well under the 200 m reject above and would not be caught by it.

(e) Carrier ambiguity. The float ambiguity that subsequently feeds integer resolution (the ambiguity-resolution document) is what remains of the *phase* residual once every other modelled and estimated term has been stripped (`StateManager::computeAmbiguitySeed`, the shared routine every phase-seeding call site invokes),

$$\widehat{N}_{r,s}^i = \Phi_{r,s}^i - \text{modelled} - \left(c\,\mathrm{d}t_r + \delta_\Phi\right) - m_w\,\text{ZWD} - \text{ISB} - g_{\text{trop}} + \alpha_s\,\widehat{I}^i, \tag{14}$$

i.e. the phase observation minus its model, receiver clock *and* phase clock δ_Φ (seed (b)), wet delay, inter-system bias, and tropospheric gradient, plus the slant ionospheric phase advance $+\alpha_s\,\widehat{I}^i$ (seed (d)): the phase leads by the same α_s the code lags, so the iono enters with the sign opposite to (7). Both the phase clock and the slant iono are removed here because they are already estimated states in the uncombined model; the one term this subtraction *cannot* remove is a receiver clock still sitting at zero on a network cold start, which is what pre-seed (f) exists to anchor. By construction this is the phase residual after removing every term the filter already estimates, so the seeded ambiguity absorbs exactly the integer cycle count plus the residual receiver/satellite phase-bias remainder, the quantity whose integer part AR recovers once the satellite phase bias is supplied and the receiver phase bias cancels under between-satellite differencing.

(f) Cold receiver-clock pre-seed (network path). Seeds (a)–(e) assume the receiver clock has already been solved, which is true on the PPP path, where the WLS clock seed of Section 1.1 runs first. The network producer does not run that WLS seed, so on the network path nothing pre-solves the receiver clock before the ambiguity of (e) is seeded: `computeAmbiguitySeed` then subtracts a clock term of zero and the entire free-running receiver-clock offset leaks uniformly into every cold ambiguity seed. The opt-in `seedColdReceiverClock` (`network.cold_clock_seed`, default off) closes this by writing the median GPS code residual, the very `gps_median` that ISB seed (a) computes and discards, into $x(\text{clock})$. It is self-limiting: it overwrites only a *cold* clock, treating any $|x(\text{clock})| >$ `kColdClockEpsilonM` $= 10^{-6}$ m as already solved and latching it untouched, and it returns `false` (retry next epoch, do *not* latch) when the epoch carries no GPS code observation, mirroring the GPS-reference contract of seed (a) (`StateInitializer::seedColdReceiverClock`).

1.3 BSSD pivot management

Between-satellite single differencing (BSSD, the BSSD document) re-expresses each constellation's ambiguities relative to a designated *pivot* satellite p_g, cancelling the receiver phase bias and rendering the SD ambiguities integer. The pivot is not arbitrary: a poor choice degrades every SD row it anchors, and an unstable choice forces costly re-referencing handovers. `src/core/filter/PivotManager.cpp` selects and maintains it; the baseline double-difference path uses a structurally analogous `src/core/filter/DdPivotManager.cpp`.

1.3.1 Lexicographic candidate ranking

Candidates are ordered *lexicographically*: a hard signal-coverage tier first, then a continuous score within the tier. Coverage dominates because the pivot must track every signal any non-pivot tracks, since a non-pivot signal the pivot does not carry is orphaned and dropped, so *any* full-coverage candidate outranks *any* partial-coverage candidate regardless of score (the lexicographic comparator in `PivotManager::buildEpochMaps`). Concretely, candidate a beats b iff its phase-signal count is higher, or the counts tie and its score is higher.

Within a coverage tier the tie-break score combines float-convergence and geometry (`PivotManager::scorePivotCandidate`):

$$\boxed{\text{score}(i) = -\min\!\big(1,\ \sqrt{v_{\max}(i)}\,\big) + \frac{\text{el}(i)}{90} - \tfrac{1}{2}\mathbb{1}[\text{el}(i) < 20^\circ]\,,} \tag{15}$$

where $v_{\max}(i)$ is the worst (largest) float-ambiguity variance among the satellite's tracked phase signals and $\text{el}(i)$ is its elevation in degrees. The first term penalises an un-converged pivot: on handover the new pivot's variance is added to every same-group SD state (the diagonal of the re-reference, $\boldsymbol{P}'(i,i) = \boldsymbol{P}(i,i) + \boldsymbol{P}(p_g,p_g) - 2\boldsymbol{P}(i,p_g)$, the diagonal of the S-transform, implemented in `include/filter/CovarianceReReference.hpp`), so a pivot whose own state is still settling injects that uncertainty into the entire constellation; the penalty saturates at -1 (one metre of equivalent 1σ ambiguity uncertainty), with a flat -1 sentinel for a candidate that has *no* state yet (the negative-`max_amb_variance` sentinel branch of that score). On the first BSSD epoch every candidate in a tier shares the sentinel, so the score collapses to elevation alone and no special first-election branch is needed. The el/90 term prefers high-elevation pivots, and the explicit $-\frac{1}{2}$ sub-20° penalty hard-discourages a low-elevation pivot whose tropospheric and multipath residuals exceed the formal stochastic model regardless of how converged its float state looks. Both score terms only ever *rank* candidates that have already cleared a hard elevation gate: a satellite below `pivot_min_el` (`getBssdPivotMinElevation`) is barred from election outright, in the freshest-peer scan,

the online pick, and the sticky-retention test, all three in `PivotManager::selectPivotCandidates`, so the $-\frac{1}{2}$ penalty only breaks ties among survivors rather than being the mechanism that keeps low satellites out. The gate falls through deliberately when *no* satellite clears the mask (a genuine code-only-float constellation): the pick reverts to the precomputed best/highest-elevation satellite so the candidate set is never emptied (the fallback from `pickExclusionAwareBest` to the precomputed `best_sat` / `highest_sat`). Sticky retention, the code's own *elevation hysteresis*, suppresses epoch-to-epoch flip-flop, but on that same *absolute* gate, not a challenger comparison: the incumbent is kept whenever it still clears `pivot_min_el`, is not excluded, and stays within the toe spread of Section 1.3.2 (the sticky branch of `selectPivotCandidates`), with no score evaluated on that branch, so a strictly better challenger cannot displace a qualifying incumbent; the score re-ranking runs only once the incumbent has *failed* the gate.

1.3.2 Ephemeris-freshness guardrail

A pivot anchored to a stale broadcast ephemeris steps *every* SD row that references it, because its extrapolated orbit and clock drift away from the fresher peers it is differenced against (observed in production as a Galileo pivot pinned to a one-hour-old I/NAV record that detonated the entire SD frame while its peers were minutes fresh). The guardrail is *spread-gated*: a candidate is barred from pivot election when its absolute ephemeris age $|t_{\mathrm{obs}} - t_{\mathrm{oe}}|$ exceeds the constellation's freshest qualified peer by more than

$$\texttt{kPivotToeSpreadMaxS} = 1200\ \mathrm{s} = 20\ \mathrm{min} \tag{16}$$

(the `freshestToeAgeAbs` scan in `PivotManager::selectPivotCandidates`, applied both at the online pick and at the sticky-retention test). Gating on the *spread* rather than an absolute age means that when every candidate is comparably fresh the ranking is untouched; only an outlier-stale candidate is displaced. The barred satellite remains usable as a non-pivot; the freshest peer always passes (its difference from itself is zero), so the candidate set is never emptied.

1.3.3 Pivot disturbance detection and handover

By the BSSD construction a held pivot's own SD ambiguity is identically zero with zero variance: it *is* the reference, so $x(p_g) = 0$ and $\boldsymbol{P}(p_g, p_g) = 0$ exactly. Any departure from that frozen state signals that the filter has, through process noise or an update, perturbed the reference and the SD frame is no longer self-consistent. `isPivotDisturbed` flags this when, for any of the pivot's phase signals,

$$\boxed{\left|x(p_g)\right| > 10^{-6} \quad \text{or} \quad \boldsymbol{P}(p_g, p_g) > 10^{-12}} \tag{17}$$

(`PivotManager::isPivotDisturbed`). The two thresholds are numerical-zero tolerances, not tuned gates: 10^{-6} m on the value and $10^{-12}\,\mathrm{m}^2$ on the variance are tight enough to catch any genuine leakage of state onto the pivot while tolerating floating-point round-off in the re-reference arithmetic. To prevent the predict step from *manufacturing* a spurious disturbance, the pivot's process noise is suppressed during prediction so $\boldsymbol{P}(p_g, p_g)$ does not grow off zero (the freeze-state / $\boldsymbol{Q}$-suppression mechanics); without that suppression every epoch's $\boldsymbol{Q}$ would trip (17) and force a needless handover.

When a handover does fire, either because the incumbent was disturbed or because ranking elected a different satellite, the outgoing pivot is classified. A *clean* handover (the old pivot's state is invertible and frame-consistent) can be re-referenced losslessly and remains smoother-inverse-referenceable, so the RTS smoother can later undo it; a *forced* or disturbed handover cannot be inverted cleanly and marks a frame discontinuity. In either case the actual covariance re-anchoring

(subtract the new pivot's row/column from each non-pivot group member, then zero the new pivot's row/column) is performed in place by `CovReReference::applyBssd` in `include/filter/CovarianceReReference.hpp`, the single shared implementation of the $\boldsymbol{T}\boldsymbol{P}\boldsymbol{T}^{\mathrm{T}}$ transform derived in the differencing document; it is *not* re-derived here. The baseline DD path differs structurally: rather than a subtract-and-freeze on a single pivot it negates the new-pivot row/column and subtracts from the others (a different S-transform), then re-keys the affected ambiguity states via `StateManager::renameAmbiguity` (`DdPivotManager::handleHandover`). Both paths share only the post-transform positive-definiteness guard (`enforcePsd`); the single-receiver user path runs the transform unguarded, since at user scale the groups are small and handovers rare enough that floating-point error in $\boldsymbol{T}\boldsymbol{P}\boldsymbol{T}^{\mathrm{T}}$ does not accumulate.